**Graded Group Assignment:**

# InClass Kaggle competition: Price Suggestion Prediction for Online Resale

in collaboration with consulting company Crunch Analytics [1]

Teacher: prof. dr. Dries F. Benoit
Teaching Assistant: Lukas De Kerpel
Course: Machine Learning
Academic Year 2025-2026

## Key practical arrangements

- **Start group assignment:** Tuesday, October 28 at 16:00
  - **First steps:** as soon as possible
    - Form a team of 4 students and subscribe in the Group section on Ufora.
    - Enter the Kaggle competition with your team via the entry link provided on Ufora. Make sure the naming convention is followed.
    - Accept the rules for the competition.
    - Create an UGent GitHub repository for your team. Make sure the naming convention is followed.
    - Create a submission on the test set.
  - **Deadline:** Wednesday, December 10 at 13:00
    - Select 2 prediction files to submit to the Kaggle competition.
    - Make sure that the GitHub repository contains all the code and the technical report.
    - Add the poster to the GitHub repository.
  - **Poster presentations:** Tuesday, December 16 in the afternoon
    - Practical details will be announced on Ufora.
    - The poster presentation will take place at the company's office.

[1] `https://www.crunchanalytics.be/`

# 1 Case study

Determining the right price for a product in **C2C (consumer-to-consumer) online marketplaces** is far from straightforward. Small differences in product description, brand, or condition can lead to substantial variations in price. For instance, on eBay or Vinted, two almost identical handbags can end up with completely different price tags. Can you tell which one typically sells for more?

**Handbag A:** "*Gucci Leather Tote, Black, excellent condition, original dust bag included.*"

**Handbag B:** "*Leather Handbag, Black, good condition.*"

Pricing becomes even more complex on large-scale resale platforms, where millions of unique products are listed every day. Product prices are influenced by multiple factors, including brand, condition, category, style, and seasonal demand. Moreover, since individual sellers determine their own prices, inconsistencies and pricing inefficiencies are common.

In this challenge, you will develop machine learning algorithms that automatically suggest fair and accurate product prices based on user-provided product information. You will be given real-world data from an online retailer, including product titles, descriptions, categories, brand names, and item conditions. Your goal is to train models capable of predicting realistic prices from these features.

Accurately predicting product prices is key for both individual sellers and platforms. It enables fairer pricing suggestions, increases user trust, and supports better decision-making across the marketplace. Reliable price prediction models also help identify overpriced or undervalued listings, understand brand-related value differences, and guide platform strategies toward more balanced and sustainable growth.

Once you have developed and validated your price suggestion model, you are encouraged to extend your analysis, for example by exploring pricing trends or price sensitivity across categories. These insights can help shape smarter, data-driven pricing strategies in C2C e-commerce.

# 2 Experimental design

## 2.1 Machine learning models

In the group assignment, you need to apply a minimum set of models from a range of chapters discussed in *An Introduction to Statistical Learning*. In addition to the minimum set, you are free to use other machine learning (ML) models, irrespective of whether they are discussed in the book. There is no upper limit to the number of models or the type of models that you can use.

The minimum set is as follows:

- Linear baselines (chapters 4 and 6)
    - Linear regression
    - Ridge regression
    - Lasso regression
- Generalized additive models (GAMs; chapter 7)

    - Smoothing splines (see below for further instructions)
- Tree-based models (chapter 8)
    - Decision tree
    - Bagging
    - Random forest
    - Gradient boosting
    - BART (this model was not explicitly treated in the lectures, but you are expected to master it via self-study of the book and the R lab)
- Support vector regression (SVR; chapter 9)
    - SVR with a linear kernel
    - SVR with a non-linear kernel
- Neural networks (chapter 10)
    - Multi-layer perceptron (MLP)

For the GAM model, you are required to select 5 continuous predictors to apply smoothing splines on. Think about which predictors are highly non-linear and would benefit most from having a smoothing spline. Find the optimal effective degrees of freedom $df_\lambda$ with cross-validation.

### 2.2 Validation strategy

A correct experimental design is crucial when benchmarking different ML models on a data set.

Model estimation:

1. For each model (except the MLP), do hyperparameter tuning on `train` set with cross-validation (CV).
2. For each model, re-train on `train` set with optimal hyperparameters and predict on the hold-out `validation` set.
3. Choose the single best-performing model based on `val` performance.
4. Retrain the best performing model on `train + val`.

Model deployment:

1. Use the best performing model to predict on the `test` set.

*General note on cross-validation:*

As discussed in the lecture on Data Preprocessing, several preprocessing techniques have a specific implementation that avoids data leakage. For example, impute missing values using the mean/median/mode from the training data. In the strict sense of CV, this means you would have a different imputation in each fold. None of the automated methods discussed in the R labs takes into account this nuance so we don't expect you to do this when implementing CV manually. It is optional but recommended to use the `caret` package in order to streamline your tuning setup.

*General note on tuning MLPs:*

Given that CV is not mandatory for MLPs, this means you will perform the validation set approach. As such, you will need to make a fourth set to measure the hold-out performance of the MLP.

# 3 Kaggle InClass Competition

Predictions on the test set are evaluated by the Kaggle platform. Given your final model's predicted price for each product in the test set, Kaggle returns the performance of your model after comparing it to the true dependent variable. The evaluation metric is the Root Mean Squared Error (RMSE).

The score returned after each submission is only based on a portion of the test set ($\approx 40\%$), which ranks your submission on the public leaderboard. However, there is a risk that the submissions are overfitted on the public leaderboard. To overcome this, when the competition ends, 2 submissions from each team are scored on the remaining test data ($\approx 60\%$), and the highest score is ranked on the private leaderboard. This is the final score and determines your ranking in the competition. Therefore, it is important to create a ML model that performs well overall, and not just on the public leaderboard.

You may select up to 2 submissions that count towards your final private leaderboard score. If 2 submissions are not selected, they will be automatically chosen based on your best submission scores on the public leaderboard.

The Kaggle team needs to be named "Team XX", where XX refers to your 2-digit group number on Ufora, e.g. 04 or 12. Make sure that you strictly adhere to this naming convention.

# 4 Project Evaluation

## 4.1 Deliverables

Develop and deliver your project in an UGent GitHub repository[2] named `ml25-teamXX`, where XX refers to your 2-digit group number on Ufora. Make sure that you strictly adhere to this naming convention. One team member creates the repository, adds their 4 collaborators and adds Prof. Benoit (`dfbenoit`), Lukas De Kerpel (`ludkerpe`), Matteo Ballegeer (`maballeg`) and Toon Van Camp (`toovcamp`). Do not make changes to the code and report in the repository after the second deadline has passed. The Kaggle competition will automatically close after this deadline. The poster should be pushed to the repository before the third deadline.

The repository needs to include:

- the R code
- the technical report
- the poster

### 4.1.1 R code

The R code should handle all relevant steps in the process of building a predictive model (i.e. use the CRISP-DM framework). As discussed in the lecture related to best coding practices, organize the code as well as possible. Imagine that you need to share your repository with the data science team of Crunch Analytics.

[2] `https://github.ugent.be/`

### 4.1.2 Technical report

Imagine that you present your findings to the data science team of Crunch Analytics, which has profound knowledge of ML concepts. Try to convince the reader that you master the ML pipeline and that your model is worth to be deployed in real life. You can think of the technical report as a blueprint for implementing the ML pipeline. If you give your report to another group, they should be able to replicate your project. You can be honest about what worked and what didn't work.

Closely follow the LaTeX template provided on Ufora and save the report as `teamXX_technical_report.pdf` in `pdf` format, with XX your 2-digit group number. On the title page, write the names of every group member and your group number. The word limit for the main body is 2500 words. This excludes title page, table of contents, references, and appendix. Although the limit is expressed in terms of words, be concise and include only figures that add sufficient value to the discussion. The same holds for the appendix.

### 4.1.3 Poster

Imagine that you present your findings to the management team of the client company, which is interested in the non-technical business side of the project. Try to convince the audience that your ML pipeline will benefit the company and that is is worth to be deployed in real life.

You have the flexibility to arrange the poster as you see fit, but the focus of the poster should be on:

- discussing any **creative features** you added to improve the model performance. Think about how you distinguished yourself from other groups.
- visualizing or presenting the **importance of individual features** in the model (e.g. a feature importance plot of your best performing tree-based method). Identify key variables that significantly influence product rate return. This will help in formulating strategic recommendations based on actionable insights.
- the **interpretability and explainability** of your model. Explain how the model makes predictions and highlight its decision-making process. Use visual aids or explanations to make it understandable to non-technical stakeholders.
- presenting **strategic insights** derived from the model's predictions. Suggest specific actions or decisions that the retail company can take to optimize product rate return. These recommendations should be based on the insights from the model and have a clear strategic focus.

Feel free to explore your artistic talents and design skills to make your poster visually captivating, as long as it aligns with the core content and objectives of this project.

To create the poster, follow the generic UGent template[3] available for MS PowerPoint. Download the poster from the Faculty of Economics and Business Administration (EB). Save the poster as `teamXX_poster.pdf` in `pdf` format, with XX your 2-digit group number.

## 4.2 Evaluation criteria

The R code and the report will be evaluated on the 6 criteria given in Table 1.The poster will be assessed on the 5 criteria given in Table 2.

[3] `https://styleguide.ugent.be/templates/print.html#poster`

| Topic | Description |
|---|---|
| Code quality | structured and comprehensive README<br>structured Github repository<br>use of coding best practices<br>readable code<br>no copy-pasting code |
| Data preprocessing and feature engineering | correct use of preprocessing techniques<br>creativity of data preprocessing/new features to improve performance<br>no data leakage in preprocessing pipeline |
| Model training | correct implementation of minimal set of ML models<br>use of more advanced ML models to improve perfomance |
| Hyperparameter tuning | correct hyperparameter tuning setup for each ML model<br>use of grid/random search<br>no data leakage in hyperparameter tuning |
| Model evaluation and experimental design | correct cross-validation strategy<br>correct data splitting<br>model selection on the validation set<br>evaluation metrics appropriate for problem<br>insights into performance and behavior of different ML models |
| Reporting | correct use of the LaTeX template<br>use of teletype<br>references and captions to figures and tables<br>high-resolution and relevant figures<br>good language and terminology use<br>ability to replicate results based on report/code |

Table 1: Evaluation criteria R code and report

| Topic | Description |
|---|---|
| Feature importance and contribution | Understanding the factors driving product rate return |
| Model interpretability and explainability | Understanding how the model makes predictions |
| Strategic insights and recommendations | Providing actionable recommendations derived from the model's predictions |
| Oral defense | Being able to effectively communicate and defend the findings |
| Poster design | Presenting a visually appealing, structured and creative poster |

Table 2: Evaluation criteria poster

### 4.3 Score distribution

The main goal of this assignment is to successfully build a ML pipeline, from data understanding to model evaluation, that minimizes the RMSE. The RMSE value of your best model has little direct impact on the grades and is intended to motivate you to develop a good performing model. Also, by means of the technical report, it is important to be able to sell your project to a technical audience.

This assignment accounts for 45% (i.e. 9 out of 20) of the total grade of this course. The following factors will be taken into consideration:

- Model performance (1 point): your ranking on the private leaderboard (see Figure 1, with $N$ groups in total).
- R code and technical report (5 points).
- Poster (3 points).

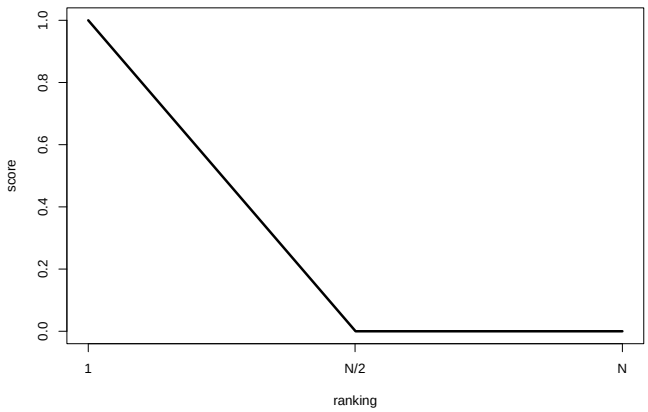



Figure 1: Score for model performance

– Peer assessment correction.

Be aware that you should have a passing grade for this group work in order to be able to pass the ML course. The project cannot be (re)done during the retake period.

# 5 Rules & Guidelines

Rules:

– Handle all relevant steps in the process of building a predictive model (i.e. use the CRISP-DM framework).

– Respect the minimum set of models outlined in section 2.1.

– The ML model should make predictions, not you hardcoding the predictions. Any violation to this rule will be considered as fraud.

– The code that you deliver needs to run error-free.

– The number of daily submissions is restricted to 3. This limitation forces you to understand what approaches might yield the highest leaderboard score. This avoids that you blindly evaluate all approaches and filter out the subset yielding the highest score. Remember that in practice, you will only have 1 such opportunity to submit the prediction (as opposed to 3 per day). The Kaggle submission system is by no means similar to Dodona; Kaggle evaluates the quality of your predictions, whereas Dodona evaluates the correctness of your code.

– Follow the LaTeX template provided on Ufora. Overleaf[4] provides an excellent guide to get started with LaTeX.

– Do not copy R output in the technical report. It is better to summarize your findings in a table.

Guidelines:

– You are encouraged to transform the variables available in the dataset. Apply whatever techniques you consider relevant.

– Although Kaggle uses the RMSE as evaluation metric, it might be interesting to check other evaluation metrics when training the model.

[4] `https://www.overleaf.com/learn/latex/Learn_LaTeX_in_30_minutes`

- Use the technical report to reflect about Occam's razor (i.e. performance-complexity trade-off).
- The experimental design and the related validation strategy are crucial when doing a ML project. Make sure that you do this in a correct way!
- When applying data preprocessing, hyperparameter tuning and/or model selection, do not introduce any kind of data leakage into your model! Introducing data leakage into the model fitting procedure makes all results unreliable and hence will have a big impact on your grades.
- Clean up messy data (e.g.`category_name` variable in the data set) using regular expressions.
- Provide a clear rationale (what, how, why) for every selected model or approach (e.g. preprocessing techniques, shortlist of models, hyperparameter settings, etc.).
- Set a `seed` value when your code includes a random aspect. This ensures that your results are replicable by your team members, as well as by us.
- Consider structuring the project repo as outlined in the lecture related to coding best practices. For example, for data pipeline, use the Layered Data Lake approach (bronze, silver, gold, etc.).
- Follow an online Git and GitHub tutorial for beginners if you are not comfortable using Git version control when doing a ML project.
- Follow best practices for writing code comments. This Stack Overflow blog[5] is an interesting source. Imagine that you need to share the GitHub repository with Crunch Analytics.
- Follow best practices for writing a README[6], a text file that introduces and explains a project. A README is written in the Markdown language[7]. Note that most README guides are targeted at professional open-source projects, and you should only include the sections that you think are relevant in this context. Give an overview of the validation set results in the README.
- Consider creating an RStudio Project[8] from the GitHub repo. This can be done with the option "create project from version control".

Good luck and have fun!

[5] `https://stackoverflow.blog/2021/07/05/best-practices-for-writing-code-comments/`

[6] `https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes`

[7] `https://guides.github.com/features/mastering-markdown/`

[8] `https://r4ds.had.co.nz/workflow-projects.html`